Technology Survey

Team Members:

- Darius Ehsani <dsehsani@ucdavis.edu>
- Joshua Bang <jjbang@ucdavis.edu>
- Nathan Rohnow <nerohnow@ucdavis.edu>
- Awaab Mirghani <almirghani@ucdavis.edu>

System Architecture: Hybrid Local + Cloud Model

After evaluating user needs and cost constraints, we propose a hybrid architecture that balances user experience, privacy, and operational costs:

Architecture Overview

Two Tier Model:

1. Local + iCloud Sync Mode [Free Model]
    - Device-Side RSS Fetching: Apple Devices fetched feeds directly using URLSession
    - Local Storage: SwiftData stores articles and metadata on-device
    - iCloud Sync (optional): Users can enable CloudKit to sync feed subscriptions, read/unread status, bookmarks, and preferences across all their Apple Devices
    - Basic Web Scraping: On-device HTML parsing for up to 5 Simple Websites
    - Update Frequency: Daily Updates via Background App Refresh
2. Cloud Backend Mode [Premium Model]
    - AWS Lambda: Server-side RSS fetching, simple web scraping, API endpoints
    - AWS Fargate: Complex web scraping with JavaScript rendering (Puppeteer/Playwright), AI batch processing for summaries
    - Amazon RDS (PostgreSQL): Centralized storage for feeds, articles, and user data
    - Amazon SQS + EventBridge: Job queue and scheduled feed updates (hourly or real-time)
    - API Gateway: REST API for app communication with rate limiting and caching
    - Amazon Bedrock (Claude): AI-powered summaries and content classification
    - iCloud Sync: CloudKit still syncs user preferences and read states across devices

Architecture Comparison

Feature	**Local + iCloud [Serverless]**	**Cloud Backend [Traditional]**
Cost to User	Free	$5-8/month
Cost to Us	$0	~$0.50-0.80/user/month
RSS Feeds	25 feeds	Unlimited feeds

Web Scraping	5 sites (on-device, simple HTML)	50+ sites (AWS, JavaScript-capable)
AI Features	❌ None	✅ Summaries, classification, filters
Cross-Device Sync	✅ Optional via CloudKit	✅ Yes via CloudKit
Update Frequency	Daily (device fetches)	Hourly or real-time
Fetch Location	On-device (battery impact)	AWS servers (no battery drain)
Scalability	Device-dependent	Auto-scales to 10K+ users
Privacy	100% local (+ optional iCloud)	Good (encrypted, no tracking)
Setup Complexity	Download app in	Sign in

Why this Architecture?

- **Zero Cost for Free Users –** Device handles fetching, optional iCloud sync is free
- **Premium Highly Profitable –** $0.50/user cost vs $6-8 revenue (10x margin)
- **User Choice –** Privacy (local) OR convenience (cloud) OR control (self-hosted)
- **Scalable –** Serverless auto-scales with demand, no capacity planning needed
- **Competitive Edge –** Only RSS reader with native web scraping + AI + three deployment modes
- **Future Ready –** Same AWS backend can support Android/web apps later

Platform/Framework: Native Swift/SwiftUI

Pros

→ Native performance and responsiveness
→ Full access to iOS APIs (widgets, shortcuts, background tasks)
→ Smallest app size and memory footprint
→ Best-in-class UI/UX with SwiftUI
→ Strong Apple ecosystem integration
→ Excellent developer tooling (XCode, Instruments)

Cons

→ iOS/macOS only (no cross-platform support)
→ Requires Swift/SwiftUI expertise
→ Cannot easily port to Android/Windows
→ Platform lock-in to Apple ecosystem

Decision: We will be using XCode and Swift/SwiftUI due to their ecosystem and performance capabilities

Technology Stack Components

Frontend (iOS/macOS Native)

Development Environment

- **IDE**: Xcode 16+
- **Language**: Swift 6.0+
- **UI Framework**: SwiftUI (primary), UIKit (fallback)
- **Minimum OS**: iOS 18+ / macOS 15+
- **Dependency Management**: Swift Package Manager (SPM)

Data & Persistence

- SwiftData
    - Local database (replaces Core Data)
    - Automatic CloudKit schema generation
    - Macro-based models
    - Built-in migration support
- CloudKit
    - iCloud container for sync
    - Private database for user data
    - CKSyncEngine for efficient sync
    - Push notifications via subscriptions

Networking

- URLSession
    - Default session for RSS feed fetching
    - Background session for updates
    - HTTP caching support
- Network Framework
    - Connection monitoring
    - Offline detection
    - WiFi vs Cellular awareness

Background Processing

- BackgroundTasks
    - BGAppRefreshTask (feed refresh every 15min-24hr)
    - BGProcessingTask (long-running tasks)

    - BGTaskScheduler (task management)

Parsing & Content

- Foundation XML
    - XMLParser (SAX-style, memory efficient)
    - XMLDocument (DOM-style, if needed)
- SwiftSoup (Third-party)
    - HTML parsing for web scraping
    - CSS selectors
    - Text extraction
- NaturalLanguage
    - On-device language detection
    - Text tokenization
    - Named entity recognition
    - Sentiment analysis

UI Components

- SwiftUI
    - Declarative UI
    - State management
    - Navigation
    - AsyncImage
- UIKit (where needed)
    - WKWebView for article rendering
    - Advanced gestures
    - Custom transitions
- WebKit
    - WKWebView for displaying articles
    - JavaScript injection support
    - Custom CSS for reader mode
- SafariServices
    - SFSafariViewController (in-app browser)
    - SFAuthenticationSession (OAuth)

Platform Integration

- WidgetKit
    - Home/Lock screen widgets
    - Timeline provider
    - Deep links
- AppIntents
    - Siri integration
    - Shortcuts support

    - Spotlight integration
- UserNotifications
    - Local notifications
    - Rich notifications (images, actions)
    - Custom categories

Third-Party Libraries (SPM)

- FeedKit: RSS/Atom/JSON Feed parsing
- SwiftSoup: HTML parsing and scraping
- Kingfisher: Image loading and caching
- Alamofire: Advanced networking (optional)
- TelemetryDeck: Privacy-focused analytics

Backend (Cloud/Premium Tier)

Compute Services

AWS Lambda

- **Runtime**: Node.js 20.x (most functions), Python 3.12 (AI/scraping)
- **Architecture**: ARM64 (Graviton2 - 20% cheaper)
- **Deployment**: AWS SAM or Serverless Framework

Essential Functions:

1. **feed-fetcher** (Node.js)
    - Fetches RSS feeds via HTTP
    - Parses XML/JSON
    - Stores articles in database
    - Memory: 512 MB, Timeout: 60s
    - Trigger: EventBridge (hourly cron)
2. **api-handler** (Node.js)
    - REST endpoints for mobile clients
    - Authentication via JWT
    - CRUD operations
    - Memory: 256 MB, Timeout: 30s
    - Trigger: API Gateway
3. **web-scraper-lite** (Python)
    - BeautifulSoup for HTML parsing
    - CSS selector-based extraction
    - Memory: 512 MB, Timeout: 120s
    - Trigger: SQS queue

Database

Amazon RDS PostgreSQL

```
Engine: PostgreSQL 16.x
Instance: db.t4g.micro ($13/month)
Storage: 20 GB gp3
Backup: 7-day retention
Encryption: AES-256
```

Essential Tables (MVP):

```
users (id, email, cognito_sub, subscription_tier, created_at)
feeds (id, url, title, last_fetched_at)
articles (id, feed_id, guid, title, content, published_at)
user_feeds (user_id, feed_id, folder, subscribed_at)
read_states (user_id, article_id, is_read, is_starred)
scraped_sites (user_id, url, selector_config)
```

Key Indexes:

- articles(feed_id, published_at DESC)
- read_states(user_id, is_read)

Why PostgreSQL:

- Relational data model fits RSS perfectly
- Full-text search built-in
- JSONB for scraper configs
- Proven, stable, familiar

Storage

Amazon S3

```
Bucket: rss-reader-prod-media
Storage Class: Standard
Cost: ~$0.50/month (100 GB)
```

What to Store:

- Article images
- OPML exports
- User avatars

Queuing

Amazon SQS

```
Queue Type: Standard
Retention: 14 days
Visibility Timeout: 300s
Dead Letter Queue: Yes (3 retries)
```

Queues (MVP):

- feed-update-queue → feed-fetcher
- scrape-queue → web-scraper-lite

Scheduling

Amazon Eventbridge

```
Rules:
  - Hourly feed updates: cron(0 * * * ? *)
  - Daily cleanup: cron(0 2 * * ? *)
```

API

Amazon API Gateway

```
Type: REST API
Throttling: 100 req/min (free), 1000 req/min (premium)
Caching: 5 min TTL
Authorization: Cognito JWT

Endpoints (MVP):
POST   /auth/login
GET    /feeds
POST   /feeds
DELETE /feeds/{id}
GET    /articles
PUT    /articles/{id}/read
POST   /sync
POST   /scrape/sites
GET    /opml/export
POST   /opml/import
```

AI (Premium Only)

Amazon Bedrock

```
Model: Claude 3.5 Haiku
Cost: $0.25/1M input tokens, $1.25/1M output tokens
Usage: Summaries (~$0.0013/article)
```

Monitoring

AWS CloudWatch

```
Logs: 7-day retention
Metrics: Lambda errors, RDS CPU, API latency
Alarms: Error rate >5%, Latency >2s
Cost: ~$10/month
```

Security

AWS Secrets Manager

- Database password
- JWT secret
- Cost: $0.40/secret/month

AWS IAM

- Least-privilege roles
- Lambda execution role
- MFA for console access

Authentication

AWS Cognito

```
Pool: rss-reader-users
Sign-in: Email, Sign in with Apple
Password: 8+ chars, complexity required
Cost: Free (<50k users)
```

Cost Breakdown

```
RDS t4g.micro:          $13/month
Lambda:                 $20/month (100k invocations)
S3:                     $1/month
SQS + EventBridge:      $2/month
API Gateway:            $10/month
CloudWatch:             $10/month
Cognito:                $0 (free tier)
Secrets Manager:        $1/month
──────────────────────────────────
Total:                  ~$57/month

Per premium user (100):  $0.57/month
Revenue ($8/user):       $800/month
Profit:                  $743/month (93% margin)
```

Additional Technical Decisions

1. **RSS/Feed Parsing:**
   - FeedKit or SwiftRSS - Native Swift RSS/Atom/JSON Feed parser
   - Support for: RSS 2.0, Atom 1.0, JSON Feed
   - Error handling for malformed feeds
   - Charset detection and encoding
2. **Web Scraping and Content Extraction**
   - Local (Free Tier)
     - SwiftSoup - HTML parsing, similar to JSoup
     - Simple CSS selectors for article extraction
     - Readability algorithm for clean article view
   - Cloud:
     - AWS Fargate + Puppeteer/Playwrite
     - Mercury Parser or custom extraction rules
     - JavaScript rendering for dynamic sites
     - Rate limiting and respect for robots.txt
3. **Image Handling & API Layer**
   - Kingfisher or Nuke - Async image downloading/caching
   - WebP support for smaller image sizes
   - Thumbnail generation for list views
   - Memory and disk cache limits
4. **Networking & API Layer**
   - Local:
     - URLSession with custom request builders
     - Network reachability monitoring

        - Request retry logic with exponential backoff
        - ETag/If-Modified-Since for efficient updates
    - Cloud:
        - AWS API Gateway REST endpoints
        - AWS Amplify Swift SDK for API calls
        - GraphQL consideration for flexible data fetching
        - Rate liming (TBD - how many request per hour / premium model)
5. **Background Processing**
    - Background App Refresh - Daily updates (free), hourly (premium)
    - BGProcessingTask for feed updates
    - BGAppRefreshTask for quick sync checks
    - Silent Push Notifications (premium) for instant updates
    - Badge count updates for unread articles
6. **Authentication & Security**
    - Free Tier:
        - Anonymous local-only mode
        - Sign in with Apple (recommended, privacy-focused)
        - CloudKit authentication (automatic with iCloud)
    - Premium Tier:
        - Amazon Cognito user pools
        - OAuth 2.0 for third-party integrations (Feedly, Inoreader)
        - API key management for web scraping services
        - Keychain storage for sensitive tokens
        - End-to-end encryption for synced data

Technology	Pros	Cons
RSS/Feed Parsing - FeedKit	✅ Pure Swift, no Obj-C bridging ✅ Supports RSS 2.0, Atom 1.0, JSON Feed ✅ Active maintenance ✅ Good error handling ✅ Type-safe parsing	❌ Larger library size vs. minimal parsers ❌ May include features you don't need ❌ Learning curve for custom extensions
RSS/Feed Parsing - SwiftRSS	✅ Lightweight ✅ Simple API ✅ Good for basic needs	❌ Less comprehensive ❌ Fewer feed format support ❌ Less active maintenance
Web Scraping - SwiftSoup (Local)	✅ Pure Swift implementation ✅ jQuery-like CSS selectors ✅ No JavaScript dependencies ✅ Works offline ✅ Low battery impact ✅ Small footprint (~100KB)	❌ Cannot handle JavaScript-rendered content ❌ Limited to static HTML ❌ Manual rule creation per site ❌ Breaks when sites change structure

		❌ No automatic content detection
Web Scraping - Puppeteer/Playwright (Cloud)	✅ Full JavaScript rendering ✅ Handles dynamic content (React, Vue, etc.) ✅ Can interact with pages (click, scroll) ✅ Screenshot capability ✅ Powerful automation	❌ Higher AWS costs ($0.10-0.20 per 1000 scrapes) ❌ Slower than static parsing ❌ Complex deployment ❌ Memory intensive ❌ Requires container management
Web Scraping - Mercury Parser (Cloud)	✅ Specialized for article extraction ✅ Pre-built rules for popular sites ✅ Cleaner output	❌ Service discontinued (must self-host) ❌ Limited site coverage ❌ No JavaScript rendering
Image Handling & Caching - KingFisher	✅ Industry standard (most popular) ✅ Excellent documentation ✅ Memory + disk caching ✅ Async image loading ✅ SwiftUI support ✅ GIF/WebP support ✅ Image processing pipeline	❌ Larger dependency (~2MB) ❌ May be overkill for simple needs ❌ Learning curve for advanced features
Image Handling & Caching - Nuke	✅ Better performance (benchmarks) ✅ Modern Swift concurrency ✅ Smaller footprint ✅ More configurable	❌ Less community adoption ❌ Fewer third-party integrations ❌ Steeper learning curve
Image Handling & Caching - SDWebImage	✅ Mature and stable ✅ Extensive platform support	❌ Objective-C based ❌ Less idiomatic Swift API ❌ Larger size
Networking & API Layer - URLSession (Local)	✅ Native iOS framework ✅ No dependencies ✅ Full control ✅ Modern async/await support ✅ Background transfer support ✅ Well documented	❌ Verbose for complex requests ❌ Manual request building ❌ No built-in retry logic ❌ Manual JSON parsing
Networking & API Layer - AWS Amplify Swift SDK	✅ Official AWS integration ✅ Built-in auth handling ✅ GraphQL support ✅ Real-time subscriptions ✅ Offline sync capabilities	❌ Heavy dependency (~10MB) ❌ Steeper learning curve ❌ Opinionated architecture ❌ Overhead for simple REST APIs
Background Processing - BackgroundTasks Framework	✅ Native iOS 13+ API ✅ Energy efficient	❌ No guaranteed execution time

	✅ System-managed scheduling ✅ Multiple task types ✅ No additional dependencies	❌ iOS may defer tasks ❌ Limited to 30 seconds (app refresh) ❌ Difficult to debug/test ❌ User can disable in settings
Background Processing - BGProcessingTask	✅ Longer execution time (minutes) ✅ Good for heavy processing ✅ Can defer until optimal conditions	❌ Even less predictable ❌ May wait hours/days ❌ Requires device charging + WiFi
Background Processing - Silent Push Notifications	✅ Immediate updates (premium) ✅ Real-time sync ✅ 30 second execution window ✅ Reliable delivery	❌ Requires server infrastructure ❌ Costs (AWS SNS) ❌ Rate limited by Apple ❌ User can disable notifications
Authenticity & Security - Sign in with Apple	✅ Required by Apple if offering other SSO ✅ Privacy-focused (email relay) ✅ Seamless UX on Apple devices ✅ No password management ✅ Built-in fraud detection	❌ Apple ecosystem only ❌ Limited user data access ❌ Cannot migrate to non-Apple platforms ❌ User can revoke at any time
Authenticity & Security - CloudKit	✅ Automatic with iCloud account ✅ Free for users ✅ No login UI needed ✅ End-to-end encryption option	❌ Requires iCloud enabled ❌ No cross-platform support ❌ Limited to Apple ecosystem ❌ Cannot access from web

Final Technology Stack Summary

Layer	Component	Technology Chosen	Free Tier (Local)	Premium Tier (Cloud)	Purpose
Platform	App Platform	Swift + SwiftUI	✅	✅	Native iOS/macOS development
IDE	Development Environment	Xcode 16+	✅	✅	Apple ecosystem tooling
UI Framework	UI	SwiftUI (UIKit fallback)	✅	✅	Modern declarative UI

Local Storage	On-device Database	SwiftData	✅ Primary storage	Used for caching	Store articles, feeds, metadata
Cloud Sync (Apple)	Device Sync	CloudKit	Optional	Sync preferences + read states	Cross-device sync
Networking (Local)	HTTP Client	URLSession	RSS fetching	Fallback calls	Network requests
Feed Parsing	RSS/Atom Parsing	FeedKit	Parse feeds locally	Used in Lambda	Read RSS/Atom/JSON feeds
Local Web Scraping	HTML Parsing	SwiftSoup	Up to 5 simple sites	❌	Extract article content from static HTML
AI / NLP (On-device)	Text Processing	NaturalLanguage	Language detection, sentiment	❌	On-device analysis
Image Handling	Image Caching	Kingfisher	✅	✅	Efficient image loading
Background Processing	Task Scheduling	BackgroundTasks Framework	Daily refresh	Hourly refresh	Background feed updates
Cloud Compute	Serverless Functions	AWS Lambda (Node.js & Python)	❌	✅ Core backend logic	RSS fetching, API handling
Advanced Scraping	Browser Automation	AWS Fargate + Puppeteer/Playwright	❌	✅	JavaScript-rendered sites
Database	Cloud Database	Amazon RDS (PostgreSQL)	❌	✅	Central user/feed/article storage
Object Storage	Media Storage	Amazon S3	❌	✅	Store images, OPML, avatars
Task Queue	Job Processing	Amazon SQS	❌	✅	Manage scraping + fetch jobs
Scheduler	Timed Tasks	Amazon EventBridge	❌	✅	Hourly feed updates
API Layer	App ↔ Server	Amazon API Gateway (REST)	❌	✅	Secure API endpoints
Authentication (Cloud)	User Accounts	AWS Cognito	Optional	Required	User login & JWT auth

AI Summaries	AI Service	Amazon Bedrock (Claude 3.5 Haiku)	❌	✅	Article summaries & classification
Monitoring	Logging & Metrics	AWS CloudWatch	❌	✅	Performance monitoring
Secrets Management	Security	AWS Secrets Manager	❌	✅	Store DB credentials & tokens
Permissions	Access Control	AWS IAM	❌	✅	Secure resource access
Notifications	Push Alerts	UserNotifications + Silent Push	Local only	Real-time premium alerts	Notify users of updates
Browser Rendering	Article View	WKWebView	✅	✅	Display article content
Analytics	Usage Metrics	TelemetryDeck	Optional	Optional	Privacy-focused analytics

Conclusion

The core MVP of our application focuses on delivering a simple, reliable, and private open RSS reader that runs primarily on the user's device. By leveraging local processing for feed fetching, article storage through SwiftData, and optional iCloud synchronization via CloudKit, the app provides a seamless reading experience without requiring a backend server. Users can subscribe to feeds, read and manage articles, and sync their preferences and read states across Apple devices while maintaining control over their data. This local architecture minimizes operational costs, reduces system complexity, and establishes a strong technical foundation. While the MVP prioritizes essential RSS functionality, the same architecture is designed to support future expansion into a premium cloud powered tier with advanced features such as large scale scraping, AI summaries, and real-time updates.

Next Steps

- **Design and finalize UI/UX**
  - Create wireframes for core screens (feed list, article view, subscriptions, settings)
  - Define navigation flow and user interactions
  - Ensure accessibility and consistent visual design using SwiftUI components
- **Strengthen understanding of required technologies and languages**
  - Deepen knowledge of Swift and SwiftUI
  - Practice using SwiftData for local persistence
  - Learn CloudKit integration for cross device syncing
  - Review RSS/Atom feed structure and parsing with FeedKit

    - Understand background tasks and app lifecycle in iOS
- **Build the MVP**
    - Implement feed subscription and storage
    - Fetch and parse RSS feeds locally
    - Display articles in a clean reader view
    - Add read/unread tracking
    - Enable optional iCloud sync for user data
    - Implement basic background refresh for feed updates
- **Testing and iteration**
    - Test on multiple Apple devices and OS versions
    - Handle malformed feeds and network errors
    - Optimize performance and memory usage
- **Prepare for future expansion**
    - Design code structure to support cloud backend integration later
    - Keep architecture modular for AI features and advanced scraping in premium tier